Practical 7

Programs on Reflections, metadata and surge generators, Managed and Unmanaged Memory.

1. Reflection

Reflection in C# is a feature that allows a program to **inspect information about types at runtime**. Using the System.Reflection namespace, we can examine classes, methods, properties, constructors, fields, and attributes without knowing all their details in advance. Reflection is commonly used in frameworks, testing tools, serialization, and dependency injection.

2. Metadata

Metadata is information that describes the structure and characteristics of a program's code. In C#, metadata contains details about classes, methods, properties, data types, access modifiers, and attributes. The .NET runtime stores this information in an assembly, and it can be accessed using Reflection.

3. Source Generators

Source Generators are a C# compiler feature that can automatically generate additional source code during compilation. They help reduce repetitive coding by generating classes, methods, properties, or other code based on the existing source code. Unlike Reflection, which mainly works at runtime, source generators work during **compile time**.

4. Managed Memory

Managed memory is memory that is automatically controlled by the **.NET Common Language Runtime (CLR)**. Objects created using new are generally stored in the managed heap, and the **Garbage Collector (GC)** automatically identifies objects that are no longer being used and releases their memory. This reduces the chances of memory leaks and makes memory management easier for C# programmers.

5. Unmanaged Memory

Unmanaged memory is memory that is **not directly controlled by the .NET Garbage Collector**. It is commonly used when interacting with operating-system APIs, native libraries, or other unmanaged code. The programmer is responsible for allocating and releasing this memory, for example by using Marshal.AllocHGlobal() and Marshal.FreeHGlobal().

1. Program on Reflection

Aim: To inspect a class, its properties, methods, and constructors at runtime using Reflection.

Steps in Visual Studio 2019

- Open **Visual Studio 2019**
- Select **Create a new project**
- Choose **Console App (.NET Core)** or **Console App (.NET Framework)**
- Name it ReflectionDemo
- Replace the code in Program.cs.

```
using System;
using System.Reflection;

class Student
{
   public int RollNo { get; set; }
   public string Name { get; set; }

   public Student()
   {
   }

   public void Display()
   {
      Console.WriteLine("Roll No: " + RollNo);
      Console.WriteLine("Name: " + Name);
   }
}

class Program
{
   static void Main(string[] args)
   {
      Type type = typeof(Student);

      Console.WriteLine("Class Name: " + type.Name);

      Console.WriteLine("\nProperties:");
      foreach (PropertyInfo property in type.GetProperties())
      {
         Console.WriteLine(property.Name);
      }

      Console.WriteLine("\nMethods:");
      foreach (MethodInfo method in type.GetMethods())
      {
         Console.WriteLine(method.Name);
      }
```

```
        Console.WriteLine("\nConstructors:");
        foreach (ConstructorInfo constructor in type.GetConstructors())
        {
            Console.WriteLine(constructor.Name);
        }

        Console.ReadLine();
    }
}
```

Expected Output

Class Name: Student

Properties:
RollNo
Name

Methods:
Display
GetType
ToString
Equals
GetHashCode

Constructors:
.ctor

2. Program on Metadata using Custom Attributes

Aim: To create and read metadata using a custom attribute.

```
using System;

[AttributeUsage(AttributeTargets.Class)]
class StudentInfoAttribute : Attribute
{
    public string Department { get; set; }
    public int Year { get; set; }

    public StudentInfoAttribute(string department, int year)
    {
        Department = department;
        Year = year;
    }
}

[StudentInfo("Information Technology", 3)]
class Student
```

```
{
   public int RollNo { get; set; }
   public string Name { get; set; }
}

class Program
{
   static void Main(string[] args)
   {
      Type type = typeof(Student);

      StudentInfoAttribute attribute =
         (StudentInfoAttribute)Attribute.GetCustomAttribute(
            type,
            typeof(StudentInfoAttribute));

      Console.WriteLine("Class Name: " + type.Name);
      Console.WriteLine("Department: " + attribute.Department);
      Console.WriteLine("Year: " + attribute.Year);

      Console.ReadLine();
   }
}
```

Expected Output

```
Class Name: Student
Department: Information Technology
Year: 3
```

3. Program on Reflection + Metadata

Program on how **Reflection reads metadata** from a class.

```
using System;
using System.Reflection;

[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)]
class InfoAttribute : Attribute
{
   public string Description { get; }

   public InfoAttribute(string description)
   {
      Description = description;
   }
}

[Info("Student information class")]
class Student
```

```
{
    [Info("Displays student information")]
    public void Display()
    {
        Console.WriteLine("Student Display Method");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Type type = typeof(Student);

        InfoAttribute classInfo =
            type.GetCustomAttribute<InfoAttribute>();

        Console.WriteLine("Class Metadata:");
        Console.WriteLine(classInfo.Description);

        MethodInfo method = type.GetMethod("Display");

        InfoAttribute methodInfo =
            method.GetCustomAttribute<InfoAttribute>();

        Console.WriteLine("\nMethod Metadata:");
        Console.WriteLine(methodInfo.Description);

        Console.ReadLine();
    }
}
```

Expected Output

```
Class Metadata:
Student information class

Method Metadata:
Displays student information
```

5. Program on Managed Memory

Aim: To demonstrate memory managed automatically by the .NET Garbage Collector.

```
using System;

class Student
{
    public string Name;
```

```
    public Student(string name)
    {
        Name = name;
    }

    ~Student()
    {
        Console.WriteLine("Student object is being destroyed.");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Student student = new Student("Rahul");

        Console.WriteLine("Student: " + student.Name);

        student = null;

        GC.Collect();
        GC.WaitForPendingFinalizers();

        Console.WriteLine("Garbage Collection completed.");

        Console.ReadLine();
    }
}
```

Output

Possible output:

```
Student: Rahul
Student object is being destroyed.
Garbage Collection completed.
```

6. Program on Unmanaged Memory

Aim: To allocate and release memory manually using unmanaged memory.

```
using System;
using System.Runtime.InteropServices;

class Program
{
    static void Main(string[] args)
    {
        int size = 20;
```

```
        IntPtr memory = Marshal.AllocHGlobal(size);
        Console.WriteLine("Unmanaged memory allocated.");
        Console.WriteLine("Memory Address: " + memory);
        Marshal.FreeHGlobal(memory);
        Console.WriteLine("Unmanaged memory released.");
        Console.ReadLine();
    }
}
```

Output

Example:

```
Unmanaged memory allocated.
Memory Address: 12345678
Unmanaged memory released.
```

The actual memory address will vary.